

Kernel trick

- $\alpha_i \neq 0$ *only* for points in the gutter, also known as support vectors
- Decision for x_j : $w x_j + b = \sum \alpha_i y_i x_i x_j + b$
- We can change the representation of x_i to be $\phi(x_i)$
- We do not need direct access to $\phi(x_i)$
 - Instead we define $k(x_i, x_j) = \phi(x_i)\phi(x_j)$
 - The function k is called kernel
- $f(x) = \sum \alpha_i y_i k(x_i, x) + b$
 - This function is nonlinear with respect to x

Advantages of the kernel trick

- The kernel trick is powerful for two reasons.
 - it allows us to learn models that are nonlinear as a function of x using convex optimization techniques that are guaranteed to converge efficiently
 - the kernel function often admits an implementation that is significantly more computational efficient than naively constructing two $\phi(x)$ vectors and explicitly taking their dot product.
 - In some cases, $\phi(x)$ can even be infinite dimensional

Gaussian kernel

- Gaussian kernel $k(\mathbf{u}, \mathbf{v}) = \mathcal{N}(\mathbf{u} - \mathbf{v}; 0, \sigma^2 \mathbf{I})$
 - Aka RBF: Radial Basis Function
- We can think of the Gaussian kernel as performing a kind of template matching.
 - A training example $\mathbf{x}$ associated with training label y becomes a template for class y .
 - When a test point $\mathbf{x}'$ is near $\mathbf{x}$ according to Euclidean distance, the Gaussian kernel has a large response, indicating that $\mathbf{x}'$ is very similar to the $\mathbf{x}$ template.
 - The model then puts a large weight on the associated training label y .
- Overall, the prediction will combine many such training labels weighted by the similarity of the corresponding training examples.

SVM limitations

- high computational cost of training when the dataset is large.
- The current deep learning renaissance began when Hinton et al. (2006) demonstrated that a neural network could outperform the RBF kernel SVM on the MNIST benchmark.

K-Nearest Neighbors

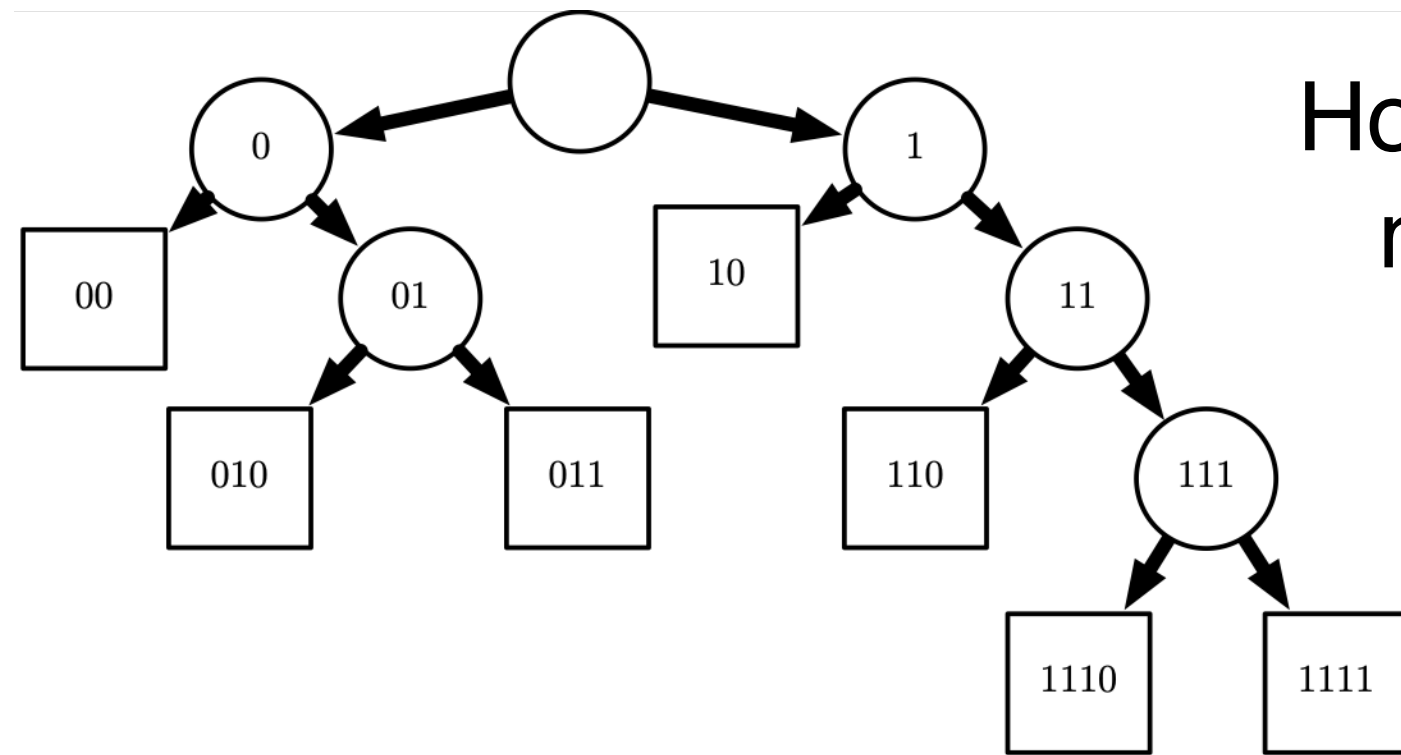
- There is not even really a training stage or learning process.
 - We find the k -nearest neighbors to $\mathbf{x}$ in the training data $\mathbf{X}$. We then return the average of the corresponding y values in the training set
- In the case of classification, we can average over one-hot code vectors
 - We can then interpret the average over these one-hot codes as giving a probability distribution over classes

K-NN limitations

- It cannot learn that one feature is more discriminative than another.
 - For example, imagine we have a regression task with $x \in \mathbb{R}^{100}$ drawn from an isotropic Gaussian distribution,
 - but only a single variable is relevant to the output
 $y = x_1$
 - Nearest neighbor regression will not be able to detect this simple pattern.

Decision Trees

How a decision tree might divide $\mathbb{R}^2$.



Each leaf requires at least one training example to define, so it is not possible for the decision tree to learn a function that has more local maxima than the number of training examples.

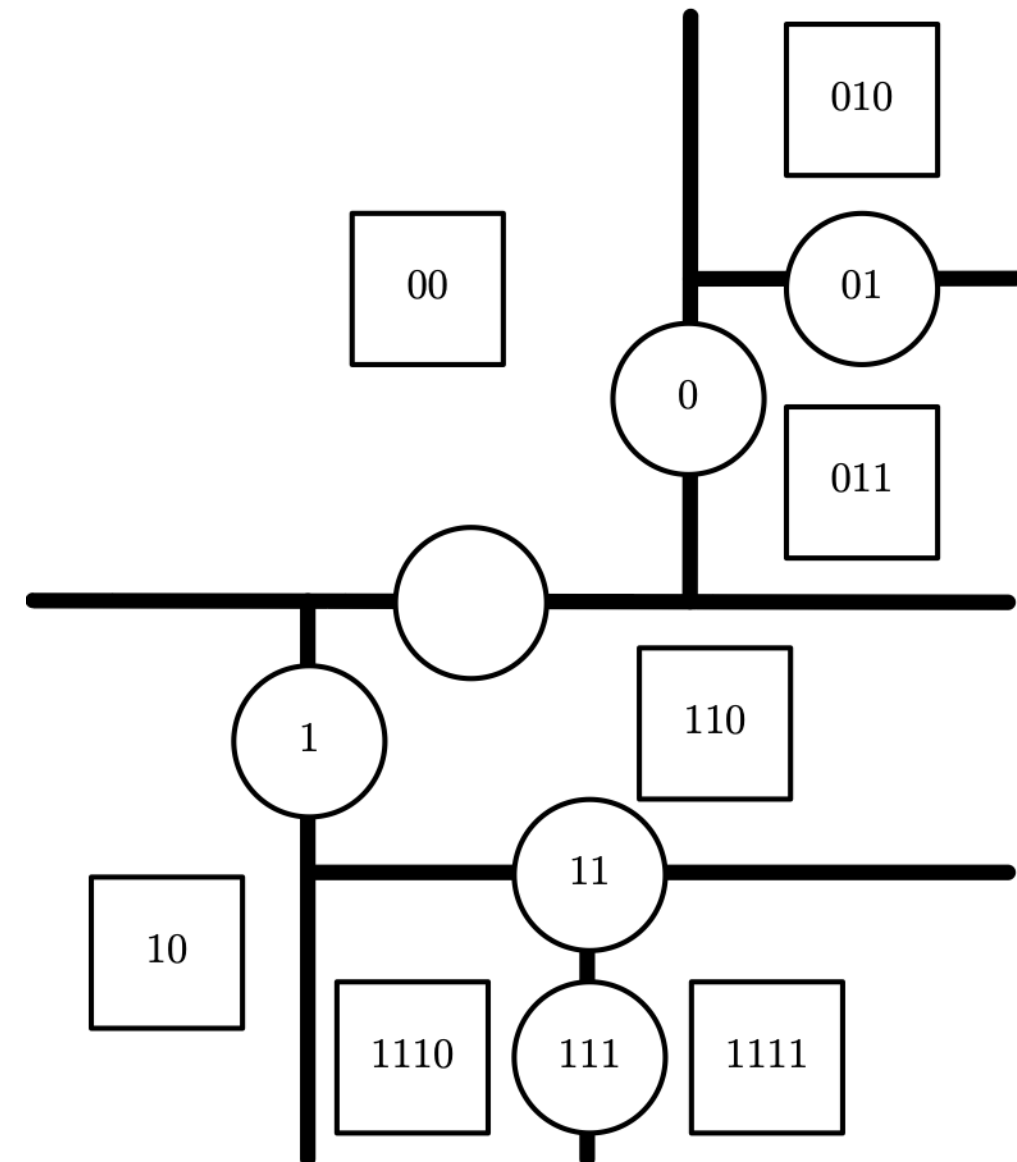


Figure 5.7

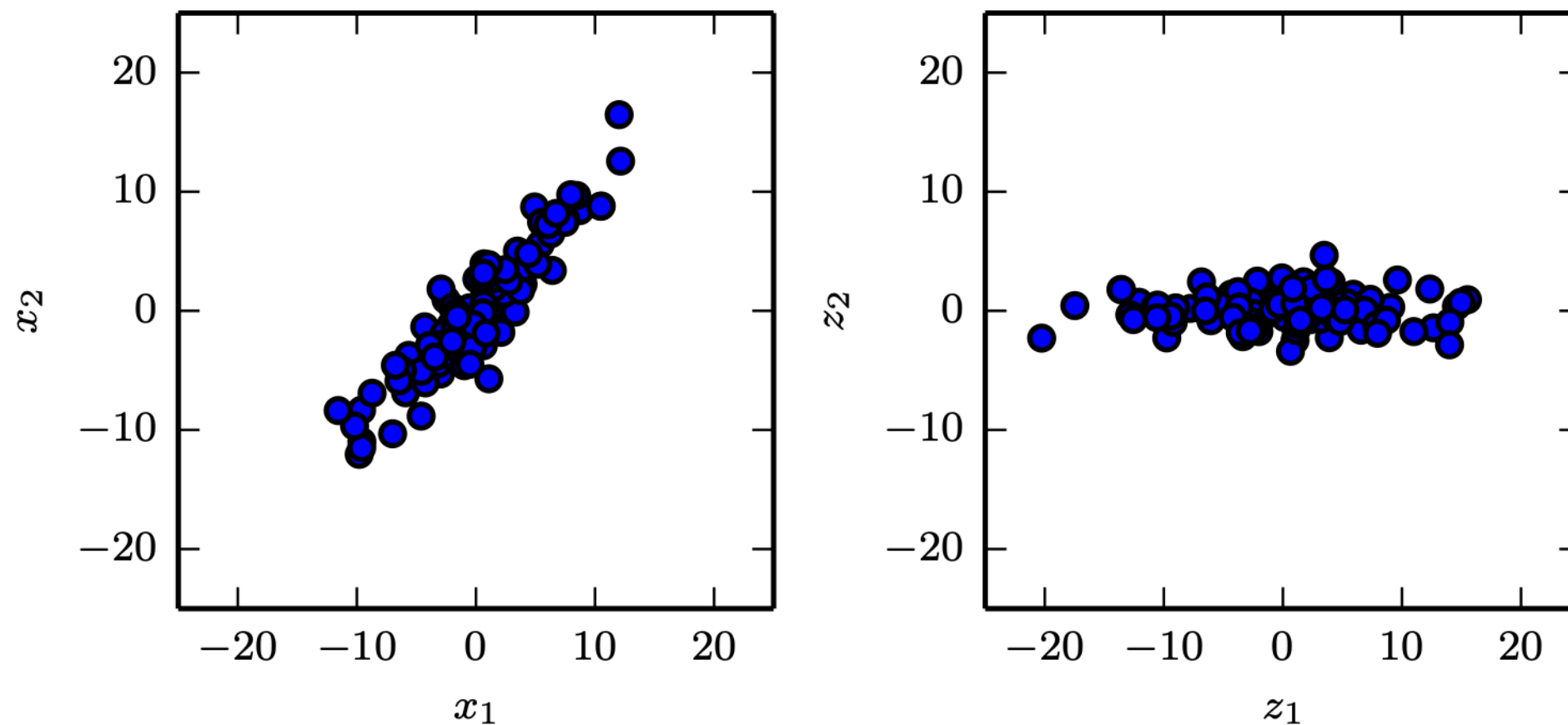
Decision trees limitations

- It struggles to solve some problems that are easy even for logistic regression.
- If we have a two-class problem and the positive class occurs wherever $x_2 > x_1$
 - the decision boundary is not axis-aligned. The decision tree will thus need to approximate the decision boundary with many splits, implementing a step function that constantly walks back and forth across the true decision function.

Unsupervised learning

- Density estimation
- Learning to draw samples from a distribution
- Learning to denoise data from some distribution
- Clustering the data into groups of related examples
- Simpler representation:
 - ❑ Lower-dimensional representation
 - ❑ Sparse representation
 - ❑ Independent representation

Principal Components Analysis



- lower dimensionality
- **no linear correlation**